DOCUMENT 09

System Architecture

How NGW works — pipeline, schema, auth, and data flow.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

Architecture Overview

NGW is a mobile-first web application backed by a Python/FastAPI service. The frontend is a React 18 SPA built with Tailwind CSS. The backend orchestrates a five-stage analysis pipeline that combines image processing, VLM inference, and a weighted consensus solver. All model calls flow through Vercel AI Gateway for cost tracking, failover, and OIDC-based authentication.

Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	18	Mobile-first SPA
Frontend	Tailwind CSS	3	Utility styling
Frontend	Vite	5	Build + HMR
Backend	Python	3.11+	Runtime
Backend	FastAPI	0.110+	REST API framework
Backend	Uvicorn	0.28+	ASGI server
Database	SQLite	3.x	Dev / embedded prod
Database	PostgreSQL	15+	Prod (optional)
AI	Vercel AI Gateway	current	VLM routing + auth
AI	Vision LLM	routed	Image interpretation
Deploy	Vercel	current	Frontend + serverless API

Auth	OIDC / Bearer Token	JWT	API + Lab access control

Database Schema

Five primary tables. Schema lives in db/database.py and is created on first run via init_db(). Migrations are run with python3 scripts/run_migrations.py.

db/database.py — session_signals (core outcomes table)

```
CREATE TABLE session_signals (
  id            TEXT PRIMARY KEY,   -- UUID v4
  session_id    TEXT NOT NULL,
  user_id       TEXT,
  pattern       TEXT NOT NULL,      -- "loop", "rembrandt", "clamshell" …
  outcome       TEXT NOT NULL,
    -- CHECK IN (nailed_it, close, failed, unknown)
  signal_source TEXT NOT NULL DEFAULT 'live',
    -- CHECK IN (live, seeded, internal, expert_review)
  include_in_learning    BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_metrics     BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_conversion  BOOLEAN NOT NULL DEFAULT TRUE,
  include_in_cohorts     BOOLEAN NOT NULL DEFAULT TRUE,
  reliability_score      REAL,
  region_scores          TEXT,   -- JSON {face,torso,bg,specular,shadow}
  degradation_flags      TEXT,   -- JSON {bw,low_res,high_contrast,...}
  gear_tier              INTEGER CHECK (gear_tier BETWEEN 1 AND 5),
  environment            TEXT,   -- studio | outdoor | hybrid
  created_at             DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

db/database.py — gold_sets and candidates tables

```
CREATE TABLE gold_sets (
  id              TEXT PRIMARY KEY,
  image_path      TEXT NOT NULL,
  pattern         TEXT NOT NULL,    -- curator-verified ground truth
  setup_details   TEXT,             -- JSON: gear, positions, power
  subject_type    TEXT,
  environment     TEXT,
  outcome_labels  TEXT,             -- JSON: expected engine output
  status          TEXT DEFAULT 'draft', -- draft|approved|archived
  curator_email   TEXT NOT NULL,
  created_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE candidates (
  id                        TEXT PRIMARY KEY,
  title                     TEXT NOT NULL,
  description               TEXT NOT NULL,
  candidate_type            TEXT NOT NULL,
  proposed_change           TEXT,  -- JSON
  status TEXT DEFAULT 'proposed',
    -- proposed|under_review|approved|rejected|applied
  estimated_success_lift    REAL,
  estimated_regression_risk REAL,
  source                    TEXT DEFAULT 'auto', -- auto|manual
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Analysis Pipeline — 5 Stages

Every reference image runs through five sequential stages. Each stage is logged independently with timing and confidence. A stage may short-circuit at the early-exit threshold (confidence ≥ 0.94).

Stage	Module	Input	Output
1. Ingestion	api/routes/shoot_match.py	Raw image file	Normalized 1024px JPEG
2. Region Extraction	engine/region_extractor.py	Normalized image	Bounding boxes + region scores
3. VLM Interpretation	engine/vlm_adapter.py	Annotated regions (3 passes)	Pattern vocabulary dicts
4. Consensus Solving	engine/solver.py	Three VLM output dicts	Resolved pattern + confidence
5. Blueprint Generation	engine/blueprint_engine.py	Pattern + user kit	Gear setup + ordered steps

VLM Integration

The VLM adapter runs three passes per image with different prompt templates: pattern-focused, modifier-focused, and geometry-focused. This reduces single-pass bias. All three response dicts go to the consensus solver.

engine/vlm_adapter.py — call structure (simplified)

```
# All model calls route through Vercel AI Gateway automatically.
# Set model via AI_MODEL_STRING env var; default: gateway-routed vision LLM.
PASS_PROMPTS = [
    PATTERN_SYSTEM_PROMPT,   # "What lighting pattern is this?"
    MODIFIER_SYSTEM_PROMPT,  # "What modifiers and shaping tools are visible?"
    GEOMETRY_SYSTEM_PROMPT,  # "Describe light positions and distances."
]

async def analyze(image_bytes, kit, model=AI_MODEL_STRING):
    results = []
    for prompt in PASS_PROMPTS:
        r = await gateway.analyze_image(
            image=image_bytes, system=prompt,
            model=model, max_tokens=1024, temperature=0.2
        )
        results.append(parse_response(r))
    return results  # List[Dict] — one per pass
```

engine/solver.py — weighted consensus (simplified)

```
# Pass weights: pass 0 = 1.0, pass 1 = 1.05, pass 2 = 1.10
# Later passes get slight boost (more context in prompt chain).
CONSENSUS_ATTRS = [
    "pattern_name", "key_angle", "key_height",
    "fill_type",    "modifier",  "fill_ratio",
]

def resolve(pass_results: List[Dict]) -> ResolvedPattern:
    out = {}
    for attr in CONSENSUS_ATTRS:
        votes = {v[attr]: 0.0 for v in pass_results if attr in v}
        for i, v in enumerate(pass_results):
            if attr in v: votes[v[attr]] += PASS_WEIGHTS[i]
        winner = max(votes, key=votes.get)
        confidence = votes[winner] / sum(votes.values())
        out[attr] = (winner, round(confidence, 3))
    return ResolvedPattern(**out)
```

Blueprint Generation & Kit Matching

The Blueprint Engine maps a resolved pattern to the user's kit through a 5-tier cascade. It stops at the highest-fidelity tier that yields a viable setup, then constructs the ordered Shoot Mode step list.

engine/blueprint_engine.py — tier cascade

```
# Tiers: 1=exact  2=close_modifier  3=alt_source  4=diy  5=ambient
for tier in range(1, 6):
    gear = kit_matcher.match(blueprint, user_kit, tier=tier)
    if gear.is_viable():
        steps = step_builder.build(
            gear=gear, role=role,
            ceiling_m=ceiling_m, room=room_dims
        )
        return Blueprint(
            pattern=resolved.pattern_name,
            confidence=resolved.confidence,
            gear=gear,   steps=steps,
            gear_tier=tier,
        )
return Blueprint.ambient_fallback(user_kit)
```

API Authentication

Method	Header / Env Var	Source	TTL
OIDC Token	Authorization: Bearer $VERCEL_OIDC_TOKEN	vercel env pull	~24 h
API Key	Authorization: Bearer $AI_GATEWAY_API_KEY	Vercel Dashboard	Manual
Lab Email Gate	x-lab-email: you@org.com	LAB_ALLOWED_EMAILS	Per request
Public Routes	(none)	N/A	N/A

OIDC VS API KEY

• On Vercel: prefer OIDC — tokens auto-refresh, no secret rotation. For local dev run: vercel link && vercel env pull .env.local

• Never commit VERCEL_OIDC_TOKEN or AI_GATEWAY_API_KEY to source control.

Full API Surface Reference

Endpoint	Method	Auth	Description
/health	GET	Public	Service liveness check
/health/db	GET	Public	Database schema + table count
/api/shoot-match	POST	Bearer	Core analysis: image recommendation
/api/upload-reference	POST	Bearer	Upload reference image (10MB max)
/api/master-modes	GET	Bearer	List photographer reference modes
/api/shoot-mode/start	POST	Bearer	Start Shoot Mode from recommendation
/api/shoot-mode/evaluate-test-shot	POST	Bearer	Compare test shot to target pattern
/api/user-data/kit	GET/POST	Bearer	Load / save user equipment kit

/api/user-data/setup	GET/ POST	Beare r	Load / save named user setups
/api/reference-library	CRU D	Beare r	Reference image library management
/api/reference-library/ingest	POST	Beare r	Ingest reference with metadata
/api/lab/gold-set	CRU D	Lab	Gold-set test image management
/api/lab/gold-set/evaluate	POST	Lab	Run engine on approved gold sets
/api/lab/candidates	CRU D	Lab	Engine improvement candidates
/api/lab/signals	POST	Lab	Ingest signals with source tags
/api/lab/signals/summary	GET	Lab	Signal hygiene counts by source
/api/lab/analyze	POST	Lab	Full-fidelity analysis + debug overlay